Lab 10.2 - Cap ol-lab-01 & Watch It Bite

KVM Virtualisation on Oracle Linux - Day 2, Module 10

Overview

Lab 10.1 controlled where ol-lab-01's resources sit - which physical cores its vCPUs run on, which NUMA node its memory comes from. This lab controls how much it's allowed to use. You'll baseline ol-lab-01 with no limits at all, apply a hard CPU quota, and watch a real workload get throttled by it in real time - then confirm the cap you set through virsh is the exact same number the kernel's cgroup controller is actually enforcing underneath.

This lab assumes ol-lab-01 exists and is running. It reuses the two-terminal pattern from Lab 12.1 - one session on ol-lab-01 to generate load, one on the host to apply the cap and watch it take effect.

Learning Objective

By the end of this lab, you will be able to:

- Explain how vcpu_period and vcpu_quota together express a CPU usage cap
- Apply a live CPU quota to a running VM with virsh schedinfo
- Observe a CPU cap's real effect on a genuine workload, not just a configuration change with no visible consequence
- Apply a disk I/O throughput cap with virsh blkdeviotune
- Verify a virsh-set cap by reading the underlying cgroup file directly, confirming libvirt and the kernel agree

Step by Step Guide

Prerequisite: Connect and Open Your Terminals

1. Terminal 1 - connect directly to ol-lab-01, since this is where you'll generate load:

sudo virsh domifaddr ol-lab-01

ssh -i ~/.ssh/id_ed25519 cloud-user@<ol-lab-01's address>

2. Install the load-generation tool used throughout this lab, if it isn't already present from Lab 12.1:

sudo dnf install -y stress-ng

3. Terminal 2 - a separate session on the host, for applying and verifying the cap:

ssh opc@<your-lab-instance-ip>

Step 1: Baseline It - Uncapped

Confirm ol-lab-01 currently has no CPU limit at all, then measure how it performs without one.

4. In Terminal 2, check the current scheduler settings:

sudo virsh schedinfo ol-lab-01

Expected result: vcpu_quota shows -1, meaning unlimited - no cap is currently applied.

5. In Terminal 1, run a short, single-vCPU CPU stress test and time it:

time stress-ng --cpu 1 --cpu-method matrixprod --timeout 20s --metrics-brief

Expected result: stress-ng completes after roughly 20 seconds and reports a bogo-ops-per-second throughput figure - note this number down, it's the uncapped baseline you'll compare against in Step 3.

6. Optionally, in Terminal 2, watch it while running:

sudo virt-top

Expected result: ol-lab-01 shows CPU usage close to a full vCPU's worth while the test runs.

Step 2: Apply a Cap with schedinfo vcpu_quota

libvirt's CPU cap works through the kernel's CFS bandwidth controller: vcpu_period defines a recurring time window (in microseconds), and vcpu_quota defines how much of that window the VM's vCPUs are allowed to actually run in. A quota of half the period caps the VM at roughly 50% of one core.

7. Check the current period, so the quota you set is proportioned correctly:

sudo virsh schedinfo ol-lab-01 | grep vcpu_period

Expected result: Typically 100000 (100,000 microseconds = 100ms) - libvirt's default.

8. Set a hard cap at 20% of one core - a fifth of the period value from above (adjust the number if your period differs from 100000):

sudo virsh schedinfo ol-lab-01 --set vcpu_quota=20000 --live --config

Expected result: Command completes with no error. --live applies it to the running VM immediately; --config makes it persist across a reboot too.

9. Confirm the setting stuck:

sudo virsh schedinfo ol-lab-01

Expected result: vcpu_quota now shows 20000 instead of -1.

Step 3: Re-run the Same Stress and Watch It Throttle

10. In Terminal 1, run the exact same stress-ng command from Step 1, unchanged:

```
time stress-ng --cpu 1 --cpu-method matrixprod --timeout 20s --metrics-brief
```

Expected result: The bogo-ops-per-second figure is now dramatically lower than the Step 1 baseline - roughly a fifth of it, matching the 20% cap - even though the test ran for the same 20 seconds and stress-ng itself wasn't told to do anything differently.

11. Optionally, watch it live in Terminal 2 while it runs:

```
sudo virt-top
```

Expected result: ol-lab-01's CPU usage now visibly caps out around 20%, rather than climbing toward 100% the way it did in Step 1 - the throttle is visibly biting in real time, not just reflected in the final throughput number.

Step 4: Read the Value Back and Confirm the Live cgroup File Agrees

virsh schedinfo is a convenient interface, but the actual enforcement happens in the kernel's cgroup controller. This step confirms the two genuinely agree, rather than trusting virsh's own report of its own setting.

12. Find ol-lab-01's cgroup path - libvirt creates one scope per VM under machine.slice:

```
CGROUP_PATH=$(sudo find /sys/fs/cgroup/machine.slice -maxdepth 1 \
  -iname "*ol*lab*01*")
echo $CGROUP_PATH
```

Expected result: A path resembling /sys/fs/cgroup/machine.slice/machine-qemu\x2d<id>\x2dol\x2dlab\x2d01.scope - libvirt's escaped naming convention for the VM.

13. Read the live cgroup v2 CPU limit file directly:

```
cat "$CGROUP_PATH/cpu.max"
```

Expected result: Two numbers separated by a space - quota then period, for example 20000 100000 - matching exactly the vcpu_quota and vcpu_period values virsh schedinfo reported in Step 2. This is the kernel's own enforcement record, not a value reported back by libvirt - the two matching is proof the cap is genuinely being enforced, not just configured.

Optional Extension: Cap Disk I/O Throughput Too

The same pattern applies to disk I/O, using virsh blkdeviotune instead of schedinfo.

14. Baseline disk throughput uncapped, writing to the /data disk from Lab 5.1:

    dd if=/dev/zero of=/data/iotest.img bs=1M count=500 oflag=direct

    Expected result: dd reports a transfer rate - note it down as the uncapped baseline.

15. Apply a 10 MB/s cap on ol-lab-01's main disk from Terminal 2:

    sudo virsh blkdeviotune ol-lab-01 vda \

      --total-bytes-sec 10485760 --live --config

16. Re-run the same dd command and compare:

    rm -f /data/iotest.img

    dd if=/dev/zero of=/data/iotest.img bs=1M count=500 oflag=direct

    Expected result: Transfer rate now capped close to 10 MB/s, regardless of how fast the underlying storage genuinely is.

17. Clean up the test file:

    rm -f /data/iotest.img

Outcomes

By the end of this lab, ol-lab-01 has a real, enforced CPU quota - not just a configuration setting, but a cap you watched a genuine workload run into and measured the difference. You've also confirmed that virsh schedinfo's reported value and the kernel's own cgroup file agree exactly, which is the actual proof a cap is working, and optionally applied the same capping pattern to disk I/O. Together with Lab 10.1's pinning, ol-lab-01 now has both its placement and its ceiling under deliberate control - the two halves of production CPU resource management.